Your individual blog project

PROGRAMMING & METHODS / ACE2627 / BERGHS

Your task is to individually build a blog. Choose a persona, subject, target audience and visual direction. The project brings together topics from the course such as HTML, CSS, JavaScript, an external API, Git and responsible use of AI assistance. SQL database and AI API features are required for Pass with distinction, but not for Pass. The aim is to build a working solution and understand how it works well enough to explain, change and test it.

Complete the tasks listed below for your target grade. Pass with distinction requires all Pass tasks and every extension task in the Pass with distinction column. Where an extension task offers alternatives, choose one of those alternatives. Completing the tasks alone does not guarantee the grade; you must also demonstrate the required understanding.

The project ends with a presentation.

Alternative theme

You may make a company marketing website instead of a blog. The same task requirements apply: include an articles or news feed and the ability to add articles. Comments stored in a database are required for Pass with distinction, but not for Pass.

What you will practise

Through the project, you will practise how to:

- Break a content need into small programming tasks.
- Use HTML and CSS to build an interface with readable text and a layout that makes sense on large and small screens.
- Read, change and debug JavaScript using functions, conditions, lists and objects.
- For Pass with distinction, store and retrieve blog content using PostgreSQL on Neon (free tier) and implement a useful AI API feature.
- Follow an external API request from input to the response displayed in the blog.
- Use Git and AI assistants while retaining control of changes and checking results.

Project tasks

Task	Pass	Pass with distinction
1. Content and navigation	Create a post listing and a few short posts. Readers can open a post and return to the listing.	-
2. Interaction	Enable adding a simple blog post. Temporary storage is enough for Pass: new posts do not need to survive a page refresh.	Enable adding comments that are stored in a database. Use PostgreSQL on Neon (free tier).
3. Layout and readability	Use readable text and a layout that makes sense on large and small screens.	-
4. JavaScript	Make a search function, article filter or a theme switcher.	-
5. External API	Fetch and display information from an external API, such as a local weather forecast from SMHI. See examples below.	-
6. Authentication	Enable signing in with a username and password, and signing out. Require users to sign in before adding a blog post.	-
7. AI API	Not required.	Use an AI API for one of the following: creating a full draft from a brief outline; reviewing a draft; generating a suitable image for the draft; or creating an automatic text summary to show along with the blog post. Use OpenRouter, Anthropic or OpenAI APIs, or an API agreed on with the teacher
8. SQL database	Not required.	Store and retrieve posts and comments using PostgreSQL on Neon (free tier).

Submission and assessment

API examples

- **SMHI weather forecast API:** fetch and display a weather forecast from SMHI. This is an example for the External API task at Pass level.
- **OpenRouter:** access AI models to suggest a blog post draft, title or summary. Build the API connection and review the result. This is an example for the AI API task required for Pass with distinction.

Documentation

Commit changes to a Git repository as you work on the project, and push to a public repository on GitHub or elsewhere. Each commit should represent a meaningful chunk of work, such as implementing authentication, storage of posts, enabling comments, displaying blog posts etc. Include a `README.md` in the project describing what has been done and what has intentionally been left out. Include instructions for running the project and configuring any required services, using placeholders for secrets.

Do not commit passwords, API keys or database credentials to the repository.

Final submission

Push your code, including the commit history, to a public repository on GitHub, GitLab or Bitbucket. Include your presentation slides in the repository.

Submit through the assignment in Canvas:

- A link to the public repository containing your code and commit history.
- Your presentation slide deck.

Assessment

Grades are **F (Fail), P (Pass) and PD (Pass with distinction)**.

- **Pass:** complete the required Pass tasks and demonstrate a basic understanding of the techniques used.
- **Pass with distinction:** complete all Pass tasks and every listed extension task, choosing one alternative wherever alternatives are offered, and demonstrate a thorough understanding of the techniques used.
- **Fail:** the requirements for Pass have not been met.

Completing features alone is not enough. You must be able to explain how your implementation works and demonstrate your understanding in the presentation and answers to questions.

Presentation

Present your website individually to the class. Briefly introduce its purpose and audience, then demonstrate its main features.

Allow **10 minutes in total**:

- **2 minutes:** demonstrate the website.
- **6 minutes:** teach one aspect of your project.
- **2 minutes:** answer questions.

Choose one aspect of your project and act as the teacher: explain how it works, assuming your audience is unfamiliar with the topic. You can choose one of the following (other topics could work, but check with the teacher first). Each alternative has a number of sample questions to help you get going.

- Authentication. How does authentication work? How does data flow between client/server? Any risks?
- Database. How does data flow between browser, server and database? How do you connect to the database? Where is it? How do you make changes to what is stored?
- AI API. How does it work? What data do you need to send? Any things you tried and failed? How did you select a model?
- External API. How does it work? What data does it need? How did you implement it?
- Theme switcher. Why use/not use? How did you implement it?
- Post/article search. Backend/frontend? Data flow between client/server/database? Which parts match the search query and how?

Prepare a few simple slides using this structure:

1. **Introduce the concept:** explain what it is, why it is useful and what the class will learn.
2. **Explain how it works:** break it into small steps with a simple diagram or example. Define unfamiliar terms.
3. **Demonstrate your process of implementing it and how your implementation works:** demonstrate the feature and walk through a small, relevant example from your own code. Explain what you did and why, and how you worked on this. What documentation did you use? How did you prompt AI? Which problems did you stumble on along the way?
4. **Recap:** summarise the main points and invite questions.

Keep slides visual and concise. Use short bullet points and minimal working code examples rather than dense text or entire files. The goal is for classmates to understand both the concept and how you applied it in your website.